

Day 8 — Advanced React

Your simple, friendly guide for today

COMING FROM	KnockoutJS / .NET (C#) → moving to React + TypeScript
TODAY'S GOAL	Optimize the app you built: share state (Context), manage complex state (useReducer), speed it up (memo/useMemo/useCallback, lazy loading), and stop crashes (error boundaries).

How to use this: Read **Section 1 (Learning Plan)** for each idea with a small example. Then read **Section 2 (Detailed Notes)** slowly, top to bottom — one read should make it click. Keep **Section 3 (Common Mistakes)** handy while you practise, and copy **Section 4 (Concept Notes)** by pen for recall & interviews.

1 Learning Plan

Today we do not build a new app. We **optimize the app we already built** in Days 3–7 (the Employee/HR portal). Same features, but now it is easier to share state, faster to render, and it won't crash to a blank white screen when something goes wrong.

Five new tools:

- **Context API** — share data without passing props through every layer.
 - **useReducer** — handle "many related updates" in one tidy place.
 - **React.memo / useMemo / useCallback** — skip work the app doesn't need to redo.
 - **React.lazy + Suspense** — load heavy screens only when needed.
 - **Error boundaries** — show a friendly fallback instead of a crash.
-

Part A — Core Topics

1. Context API

Right now, to get the current user (or the employee list) to a deep child, you pass it as a prop through every component in between. That is called **prop drilling** — annoying and easy to break. **Context** lets you put a value in one place (a **Provider**) and read it anywhere below with `useContext`, skipping the middle layers. Type the value, and write a small `useX()` hook that **throws a clear error** if someone forgets the Provider.

```
// UserContext.tsx
import { createContext, useContext, type ReactNode } from "react";

interface Employee { id: number; name: string; }

// The value the context will share
interface UserContextValue { currentUser: Employee | null; }

// Start as null so we can detect "used with no Provider"
const UserContext = createContext<UserContextValue | null>(null);

export function UserProvider({ children }: { children: ReactNode }) {
  const currentUser: Employee = { id: 1, name: "Asha" };
  return (
    <UserContext.Provider value={{ currentUser }}>
      {children}
    </UserContext.Provider>
  );
}

// Typed hook: throws if there is no Provider above it
export function useCurrentUser(): UserContextValue {
  const ctx = useContext(UserContext);
  if (!ctx) throw new Error("useCurrentUser must be used inside <UserProvider>");
  return ctx;
}
```

- **.NET / KnockoutJS:** Like **dependency injection / a shared service** — register once, use it where you need it, instead of threading it through every constructor.
- **You'll be able to...** share the current user or the employee list app-wide, with zero prop drilling.

2. useReducer

The employee list has many related updates: **add, update, remove**. Doing all that with scattered `useState` calls gets messy. `useReducer` puts all the update rules in ONE function — `reducer(state, action)`. You `dispatch` an action (a plain object that says what happened), and the reducer returns the next state. A **discriminated union** (Day 2!) types each action, so TypeScript forces you to handle every case.

```
// useEmployees.ts
import { useReducer } from "react";

interface Employee { id: number; name: string; salary: number; }

// Discriminated union: each action has a literal "type" tag
type Action =
  | { type: "add"; employee: Employee }
  | { type: "update"; id: number; changes: Partial<Employee> }
  | { type: "remove"; id: number };

function reducer(state: Employee[], action: Action): Employee[] {
  switch (action.type) {
    case "add":
      return [...state, action.employee];
    case "update":
      return state.map((e) =>
        e.id === action.id ? { ...e, ...action.changes } : e
      );
    case "remove":
      return state.filter((e) => e.id !== action.id);
  }
}

export function useEmployees() {
  const [employees, dispatch] = useReducer(reducer, []);
  // dispatch({ type: "add", employee: { id: 1, name: "Asha", salary: 50000 } });
  return { employees, dispatch };
}
```

- **.NET / KnockoutJS:** Like a small **state machine** or a **Redux-style store** — an action goes in, the next state comes out, all rules in one place.
- **You'll be able to...** manage add/update/remove with one predictable, fully-typed reducer instead of many `setState` calls.

3. React.memo

By default, when a parent re-renders, **all its children re-render too** — even if their props didn't change.

React.memo wraps a component so React **skips** re-rendering it when its props are the same as last time (compared by reference — see referential equality below). Perfect for rows in a long employee table.

```
// EmployeeRow.tsx
import { memo } from "react";

interface RowProps { name: string; salary: number; }

// Only re-renders when name or salary actually change
const EmployeeRow = memo(function EmployeeRow({ name, salary }: RowProps) {
  return (
    <tr>
      <td>{name}</td>
      <td>{salary}</td>
    </tr>
  );
});

export default EmployeeRow;
```

- **.NET / KnockoutJS:** Like **caching a rendered result** and reusing it until the inputs change.
- **You'll be able to...** stop list rows from redrawing when nothing about them changed.

4. useMemo

useMemo caches the **result of an expensive calculation** — like a filtered/sorted employee list or a total payroll. React only recomputes it when a value in its **dependency array** changes; otherwise it hands back the last result.

```
// Payroll.tsx
import { useMemo } from "react";

interface Employee { id: number; name: string; salary: number; }

function Payroll({ employees }: { employees: Employee[] }) {
  // Recomputes only when `employees` changes, not on every render
  const total = useMemo(
    () => employees.reduce((sum, e) => sum + e.salary, 0),
    [employees]
  );
  return <p>Total payroll: {total}</p>;
}
```

- **.NET / KnockoutJS:** Like a `ko.computed` (or a cached computed property) that re-evaluates only when its inputs change.
 - **You'll be able to...** compute totals or a sorted list without redoing the work on every keystroke.
-

5. useCallback

Every render creates **brand-new function objects**. If you pass a function to a `React.memo` child, or list it in a `useEffect` dependency array, a *new* function every render **defeats the optimization** (the child re-renders anyway; the effect re-fires). `useCallback` keeps the **same function identity** between renders until its dependencies change.

```
// EmployeeSearch.tsx
import { memo, useCallback, useState } from "react";

const EmployeeList = memo(function EmployeeList({
  onSelect,
}): {
  onSelect: (id: number) => void;
}) {
  return <button onClick={() => onSelect(1)}>Pick #1</button>;
});

function EmployeeSearch() {
  const [query, setQuery] = useState("");

  // Stable identity, so the memoized <EmployeeList> does not re-render needlessly
  const handleSelect = useCallback(
    (id: number) => console.log("selected", id, query),
    [query]
  );

  return (
    <
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
      <EmployeeList onSelect={handleSelect} />
    </>
  );
}
```

- **.NET / KnockoutJS:** Like **reusing the same delegate** instead of allocating a fresh lambda on every call.

- **You'll be able to...** hand stable callbacks to memoized children so `React.memo` actually pays off.
-

6. React.lazy + Suspense

`React.lazy` loads a component's code **only when it is first shown**, splitting it into its own downloaded file (a "chunk"). While that code is downloading, `<Suspense fallback={...}>` shows a loader. This is called **code splitting** — it keeps the first page load small and fast.

```
// Dashboard.tsx
import { lazy, Suspense } from "react";

// Its code loads only when it first renders (a separate JS chunk)
const EmployeeChart = lazy(() => import("./EmployeeChart"));

function Dashboard() {
  return (
    <Suspense fallback={<p>Loading chart...</p>}>
      <EmployeeChart />
    </Suspense>
  );
}
```

- **.NET / KnockoutJS:** Like **lazy-loading an assembly/module on demand** instead of loading everything at startup.
 - **You'll be able to...** load a heavy screen (like a chart) only when the user opens it, so the app starts faster.
-

7. Error boundaries

An **error boundary** catches errors thrown **while its children render**, and shows a **fallback UI** instead of a blank white screen. It is the **one legitimate modern use of a class component**: it uses

`getDerivedStateFromError` (flip to the fallback) and `componentDidCatch` (log the error). Important: it only catches **render** errors — not clicks or async (more on that in Part B).

```
// ErrorBoundary.tsx
import { Component, type ErrorInfo, type ReactNode } from "react";

interface Props { children: ReactNode; }
interface State { hasError: boolean; }

class ErrorBoundary extends Component<Props, State> {
  state: State = { hasError: false };

  // Runs during render when a child throws - switch to the fallback
  static getDerivedStateFromError(): State {
    return { hasError: true };
  }

  // Side effects only: log the error somewhere
  componentDidCatch(error: Error, info: ErrorInfo): void {
    console.error("Caught by boundary:", error, info.componentStack);
  }

  render(): ReactNode {
    if (this.state.hasError) return <h2>Something went wrong.</h2>;
    return this.props.children;
  }
}

export default ErrorBoundary;
```

- **.NET / KnockoutJS:** Like a **try/catch wrapped around your UI render**, showing a friendly error page instead of crashing.
- **You'll be able to...** keep one broken widget from taking down the whole app.

Part B — Extra Topics (go a little deeper)

When to optimize — measure first, don't guess

Do **not** sprinkle `memo` / `useMemo` / `useCallback` everywhere "just in case." They are **not free**: they use memory, they run a comparison every render, and they add code noise. React is already fast for most screens. **First measure** with the **React DevTools Profiler** — a browser tab that records which components re-rendered and how long each took. Optimize only the parts the Profiler shows are actually slow. Premature optimization usually makes code harder to read for zero real speed-up.

Context + `useReducer` = a mini-Redux (and the re-render pitfall)

Pair them: `useReducer` holds your app-wide state, **Context** shares it. That is basically a **tiny Redux with no extra library**.

The pitfall: when a Context value changes, **every** component that reads that context re-renders — even ones that only cared about a *different* part of it. Two fixes:

1. **Split into smaller contexts** (e.g. a `UserContext` and a separate `EmployeesContext`), so a change in one doesn't wake up consumers of the other.
2. **Split state and dispatch** into two contexts. Components that only *send* actions read the dispatch context — and because `dispatch` never changes identity, they don't re-render when the state changes.

```
// EmployeesProvider.tsx
import { createContext, useReducer, type ReactNode, type Dispatch } from "react";

interface Employee { id: number; name: string; }
type Action =
  | { type: "add"; employee: Employee }
  | { type: "remove"; id: number };

function reducer(state: Employee[], action: Action): Employee[] {
  switch (action.type) {
    case "add": return [...state, action.employee];
    case "remove": return state.filter((e) => e.id !== action.id);
  }
}

// Two separate contexts: reading state and sending actions are split apart
const StateCtx = createContext<Employee[]>([]);
const DispatchCtx = createContext<Dispatch<Action> | null>(null);

export function EmployeesProvider({ children }: { children: ReactNode }) {
  const [state, dispatch] = useReducer(reducer, []);
  return (
    <StateCtx.Provider value={state}>
      <DispatchCtx.Provider value={dispatch}>{children}</DispatchCtx.Provider>
    </StateCtx.Provider>
  );
}
```

Referential equality — why `{}` `!==` `{}` and why it matters

`===` on objects, arrays, and functions compares the **reference** (their address in memory), **not the contents**. So `{}` is never equal to another `{}`, and `[]` is never equal to another `[]`.

```
// referential-equality.ts
const a = {};
const b = {};
console.log(a === b); // false - two different objects in memory
console.log(a === a); // true - same reference

// This is why a fresh object created during render breaks React.memo:
// <Child style={{ color: "red" }} /> // {} is brand-new every render
```

This is the whole reason `React.memo`, `useMemo`, `useCallback`, and `useEffect` dependency arrays behave the way they do. A new object/array/function made **during render** looks "different" every time. Fixes: cache it with `useMemo` / `useCallback`, or move constants **outside** the component so they're created once.

Error boundaries only catch RENDER errors

An error boundary catches errors during **rendering**, in **lifecycle methods**, and in child **constructors**. It does **NOT** catch:

- **event handlers** (e.g. `onClick`),
- **async code** (`setTimeout`, `fetch`, `await`),
- errors thrown in the boundary itself.

For those, use a normal `try/catch` and handle the failure yourself (show a toast, set an error state, etc.).

```
// SaveButton.tsx
declare function saveEmployee(): Promise<void>;

function SaveButton() {
  async function handleSave() {
    try {
      await saveEmployee();
    } catch (err) {
      // An error boundary will NOT catch this - handle it here
      console.error(err instanceof Error ? err.message : String(err));
    }
  }

  return <button onClick={handleSave}>Save</button>;
}
```

Lighter option: the `react-error-boundary` library gives you a ready-made `<ErrorBoundary>` component with a `fallbackRender`, an `onError` logger, and a **reset** button (plus a `useErrorBoundary`

hook to trigger the fallback from async code) — so you write far less class boilerplate. Worth knowing; the hand-written class above is still good to understand *why* it works.

Quick .NET / KnockoutJS map

React (Day 8)	Familiar idea
Context API	Dependency injection / a shared service
useReducer	A state machine / a Redux store
React.memo / useMemo / useCallback	Caching a computed value (or a <code>ko.computed</code>)
React.lazy + Suspense	Lazy-loading a module on demand
Error boundary	try/catch wrapped around your UI render

Where this fits: Day 8 is the **last learning day**. Next is Day 9 (getting the app production-ready) and Day 10 (assessment). Everything here is about making the existing Employee portal **cleaner to share state, faster, and harder to crash** — not about adding new features.

Detailed Notes — Read Once, Understand Everything

The big idea first

You already built a working Employee portal over the last 7 days. Today is **not** about new screens. Today is about making that same app **cleaner, faster, and tougher**. Everything falls into three buckets:

1. **SHARING state** without passing props through every level — this is the **Context API**. (Think of it like **dependency injection** in .NET: instead of handing a service down through every constructor, any component just *asks* for it.)
2. **MANAGING complex state** cleanly, when many pieces change together — this is **useReducer**. (Think of it like a small **state machine**, or a mini **Redux**: one function decides every change.)
3. **Making the app FAST and RESILIENT** — `React.memo`, `useMemo`, `useCallback` (all a form of **caching**, like memoizing an expensive method result in C#), plus `React.lazy` for loading code only when needed, and **error boundaries** so one broken component doesn't blank the whole screen.

And the **golden rule** for today: **optimize only when you have *measured* a real problem**. Most apps are already fast. Adding caching everywhere "just in case" makes the code harder to read and can even make it slower. Measure first, then fix.

1. The prop drilling problem — and the Context API

Prop drilling means passing a value down through many components that don't even use it — they just hand it to the next child. It's like a parcel passed through five people to reach the sixth. Annoying, and every middle component's signature gets cluttered.

React's fix is the **Context API**. You create a "shared box", put a value in it once at the top, and any component below can read it directly — no matter how deep. This is very close to a **shared service via dependency injection** in .NET.

Three steps: **create** the context, **wrap** your tree in a Provider, **read** it with `useContext`. And always give it a **typed** value.

```

import { createContext, useContext, useState, type ReactNode } from "react";

interface User {
  id: number;
  name: string;
  role: "admin" | "hr" | "employee";
}

interface AuthContextValue {
  user: User | null;
  login: (user: User) => void;
  logout: () => void;
}

// createContext needs a starting value. We use `undefined` and
// check for it inside the custom hook below.
const AuthContext = createContext<AuthContextValue | undefined>(undefined);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);

  const login = (u: User) => setUser(u);
  const logout = () => setUser(null);

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

// Custom hook: read the context, and THROW if it's used with no Provider.
// This turns a silent bug into a loud, clear error.
export function useAuth(): AuthContextValue {
  const ctx = useContext(AuthContext);
  if (ctx === undefined) {
    throw new Error("useAuth must be used inside <AuthProvider>");
  }
  return ctx;
}

```

Now any component just calls `const { user, logout } = useAuth();`. No drilling. The little `useAuth()` wrapper is a nice pattern: it gives you a clean name, a solid type (never `undefined`), and a friendly error if you forget the Provider. You'd write a matching `useEmployees()` the same way.

2. The Context performance pitfall

Here's the catch. **When the context *value* changes, every component that reads that context re-renders** — even the ones that only care about a part that didn't change. If you cram *everything* (the current user, the whole employee list, the theme, filters) into one giant context, a tiny change makes the whole app repaint.

Two simple habits fix this:

- **Keep contexts small and split by topic.** An `AuthContext`, a separate `EmployeesContext`, a separate `ThemeContext`. Changing the theme shouldn't re-render the employee table.
- **Pair a context with `useReducer`** (next section) so the *update functions* you share stay stable, and only the data that truly changed causes re-renders.

Rule of thumb: one context = one concern.

3. `useReducer` — for complex, related state

`useState` is perfect for one simple value. But when you have **many pieces of state that change together** through **many different actions** (add, remove, select, clear, load...), a pile of separate `useState` calls gets messy and error-prone.

`useReducer` gathers all of that into **one function** that decides every change. That function is a **reducer**: a **pure function** with the shape `(state, action) ⇒ newState`. "Pure" means same inputs always give the same output and it changes nothing outside itself (no API calls, no mutating). This is basically a **state machine** — or the reducer at the heart of **Redux**, if you've heard of it.

You describe the possible actions with a **discriminated union** (Day 2 pays off here!): each action has a `type` tag, and TypeScript uses that tag to know exactly what other fields are allowed.

```

import { useReducer } from "react";

interface Employee {
  id: number;
  name: string;
  active: boolean;
}

interface State {
  employees: Employee[];
  selectedId: number | null;
}

// Discriminated union: `type` is the tag that separates the actions.
type Action =
  | { type: "add"; employee: Employee }
  | { type: "remove"; id: number }
  | { type: "select"; id: number }
  | { type: "clearSelection" };

// A reducer is PURE and must return a NEW state object (never mutate).
function employeesReducer(state: State, action: Action): State {
  switch (action.type) {
    case "add":
      return { ...state, employees: [...state.employees, action.employee] };
    case "remove":
      return {
        ...state,
        employees: state.employees.filter((e) => e.id !== action.id),
      };
    case "select":
      return { ...state, selectedId: action.id };
    case "clearSelection":
      return { ...state, selectedId: null };
    default:
      return state;
  }
}

function EmployeeManager() {
  const [state, dispatch] = useReducer(employeesReducer, {

```

```

    employees: [],
    selectedId: null,
  });

  // You don't change state directly – you "dispatch" (send) an action,
  // and the reducer decides the next state.
  const addAya = () =>
    dispatch({ type: "add", employee: { id: 1, name: "Aya", active: true } });

  return <button onClick={addAya}>Add (total: {state.employees.length})</button>;
}

```

Notice `dispatch({ type: "select", id: 3 })` — because of the discriminated union, TypeScript *forces* you to include `id` for `"select"`, and *forbids* it for `"clearSelection"`. Typos become compile errors.

Prefer `useReducer` over `useState` when: state has several related fields, or the next value depends on the previous one, or you have many distinct update types. **Immutability rule (again):** never edit the old state in place — always **return a brand-new object/array** (`{ ...state, ... }`, `[...list]`). React decides whether to re-render by comparing the old and new state, so a fresh object is essential.

4. Referential equality — the "why" behind all the memo tools

Before performance tools make sense, you need one idea. In JavaScript:

- **Primitives** (number, string, boolean) compare **by value**. Same value = equal.
- **Objects, arrays, and functions** compare **by reference** (by identity). Two things with the *same contents* are still **not equal** if they're two separate objects.

```

// Primitives compare by VALUE:
const sameNumber = 5 === 5;           // true
const sameText = "hr" === "hr";       // true

// Objects / arrays / functions compare by REFERENCE (identity).
// Same contents, but two different objects:
const listA = [1, 2, 3];
const listB = [1, 2, 3];
const equal = listA === listB;         // false

```

Why does this matter? Because **every time a component renders, any object, array, or function you create inside it is brand-new** — a different reference than last render, even if it looks identical. So a child that receives `style={{ color: "red" }}` or `onClick={() => ...}` sees a "new" prop every single time. That's exactly the problem the next three tools solve.

5. React.memo — skip re-rendering a child

`React.memo` wraps a component and says: "if my props are the **same** as last time, don't re-render me." It does a **shallow compare** — it checks each prop with `===` (one level deep).

```
import { memo } from "react";

interface RowProps {
  name: string;
  onSelect: (name: string) => void;
}

// Skips re-render when `name` and `onSelect` are each === the previous value.
const EmployeeRow = memo(function EmployeeRow({ name, onSelect }: RowProps) {
  console.log("render", name);
  return <li onClick={() => onSelect(name)}>{name}</li>;
});

export default EmployeeRow;
```

Big catch: `React.memo` only helps if the props you pass are actually **stable**. If the parent hands it a fresh `onSelect` function or a fresh object on every render (remember Section 4), the shallow compare always says "different", and `memo` does nothing. So `React.memo` on the child usually needs `useCallback` / `useMemo` on the parent to be worth it.

6. useMemo — cache an expensive computed value

`useMemo` **remembers a calculated value** between renders and only **recomputes it when its dependencies change**. It's caching, exactly like memoizing a slow method result in C#. Great for expensive filtering, sorting, or totals over a big list.

```

import { useMemo, useState } from "react";

interface Employee {
  id: number;
  name: string;
  salary: number;
}

function PayrollList({ employees }: { employees: Employee[] }) {
  const [search, setSearch] = useState("");

  // Filter + sort runs ONLY when `employees` or `search` change,
  // not on every unrelated re-render.
  const visible = useMemo(() => {
    return employees
      .filter((e) => e.name.toLowerCase().includes(search.toLowerCase()))
      .sort((a, b) => b.salary - a.salary);
  }, [employees, search]);

  return (
    <div>
      <input value={search} onChange={(e) => setSearch(e.target.value)} />
      <ul>
        {visible.map((e) => (
          <li key={e.id}>{e.name}</li>
        ))}
      </ul>
    </div>
  );
}

```

The `[employees, search]` at the end is the **dependency array** — the list of things that, when changed, force a recompute. Get it right, or you'll show stale data.

7. useCallback — keep a function's identity stable

`useCallback` is the same idea as `useMemo`, but for a **function**. It hands back the **same function reference** across renders until its dependencies change. In fact, `useCallback(fn, deps)` is literally shorthand for `useMemo(() => fn, deps)`.

```
import { useCallback, useState } from "react";

function EmployeeManager() {
  const [count, setCount] = useState(0);

  // Without useCallback this would be a NEW function every render,
  // which would break React.memo on any child it's passed to.
  const handleSelect = useCallback((name: string) => {
    console.log("selected", name);
  }, []); // empty deps => the same function forever

  return <button onClick={() => setCount(count + 1)}>Clicked {count}</button>;
}
```

You reach for `useCallback` in two situations: (1) you pass a callback down to a `React.memo` child and want the memo to actually work, or (2) a function is listed in a `useEffect` dependency array and you don't want the effect firing on every render.

8. When NOT to optimize

Memoization is **not free**. `useMemo` and `useCallback` cost a little memory and a little comparison work on every render, and they add noise to your code. If a value is cheap to compute or a component is cheap to render, wrapping it can be **slower and harder to read** than just doing the work.

So the workflow is:

1. Build it simply first, with plain `useState` and no memo.
2. If it feels slow, **measure** with the **React DevTools Profiler** — it shows you which components re-render and how long they take.
3. Only then add `React.memo` / `useMemo` / `useCallback` to the specific hot spot the Profiler pointed at.

Measure, then optimize. Never the other way around.

9. React.lazy + Suspense — load code only when it's needed

By default your whole app is bundled into one big JavaScript file the user downloads up front. **Code splitting** breaks off heavy or rarely-used parts (a big Reports page, a chart library) into separate chunks that download **on demand**. `React.lazy` does the splitting; `Suspense` shows a fallback (a spinner, "Loading...") while that chunk arrives.

```
import { lazy, Suspense } from "react";

// This page's code downloads only the first time it's actually shown.
const ReportsPage = lazy(() => import("./ReportsPage"));

function App() {
  return (
    <Suspense fallback={<p>Loading reports...</p>}>
      <ReportsPage />
    </Suspense>
  );
}
```

This pairs beautifully with **Day 6 routing**: lazy-load each route so the login screen loads instantly and the heavy dashboard only downloads once the user goes there. Faster first load, smaller initial download.

10. Error boundaries — one crash shouldn't blank the whole app

If a component throws an error **during render**, React unmounts the entire app and the user sees a white screen. An **error boundary** catches that error, shows a friendly fallback UI instead, and keeps the rest of the app alive.

This is the **one place modern React still needs a class component**. It uses two special methods: `static getDerivedStateFromError` (set the "show fallback" state) and `componentDidCatch` (log the error somewhere).

```

import { Component, type ErrorInfo, type ReactNode } from "react";

interface Props {
  children: ReactNode;
  fallback: ReactNode;
}

interface State {
  hasError: boolean;
}

class ErrorBoundary extends Component<Props, State> {
  state: State = { hasError: false };

  // Runs when a child throws during render: switch to the fallback.
  static getDerivedStateFromError(_error: Error): State {
    return { hasError: true };
  }

  // Runs after: the right spot to log to your monitoring service.
  componentDidCatch(error: Error, info: ErrorInfo) {
    console.error("Caught by boundary:", error, info.componentStack);
  }

  render() {
    if (this.state.hasError) {
      return this.props.fallback;
    }
    return this.props.children;
  }
}

export default ErrorBoundary;

```

Wrap any risky part of the tree:

```

<ErrorBoundary fallback={<p>Something went wrong. Please refresh.</p>}>
  <EmployeeDashboard />
</ErrorBoundary>

```

Very important limit: error boundaries only catch errors that happen **during render** (and in lifecycle methods). They do **NOT** catch errors in **event handlers** (a button's `onClick`) or in **async code** (a failed `fetch` in `useEffect`). For those, use ordinary `try/catch` and set some error state yourself. If you'd rather not hand-write the class, the small `react-error-boundary` library gives you a ready-made `<ErrorBoundary>` with a `FallbackComponent` and a reset button.

One-line summary

Day 8 = share state cleanly (Context), manage complex state predictably (useReducer), and make the app fast (memo / useMemo / useCallback) and resilient (lazy + Suspense + error boundaries) — but only optimize after you've measured a real problem.

3 Common Mistakes to Avoid

These are the traps most people fall into with Context, `useReducer`, memoization, lazy loading, and error boundaries. Read the ❌, then do the ✅.

1. Reaching for Context too early (or one giant "everything" Context)

Context is for a *few* values many components need (like the logged-in user, theme, or language). It is **not** a replacement for passing normal props. And when the Context value changes, **every** component that reads it re-renders.

- ❌ Wrap the whole app in Context on day one, just to avoid passing one prop two levels down.
- ❌ Put `user`, `theme`, and a big `notifications` array in **one** Context. Now changing the theme re-renders every notification consumer too.
- ✅ First try **lifting state** only as far up as the components that share it (Day 4). Use Context only when props get painful (deep "prop drilling").
- ✅ **Split** Context by how often each value changes. Rarely-changing things (theme, current user) go in their own Context, separate from fast-changing things.

C#/.NET link: a Context is like a **shared service injected via DI** — handy, but you don't register *everything* as one giant service. Keep services small and focused.

```
// ❌ one giant context: change the theme → the whole app re-renders
const AppContext = createContext<{
  user: User;
  theme: Theme;
  notifications: Notification[];
} | null>(null);

// ✅ split by "how often does this change?"
const UserContext = createContext<User | null>(null); // rarely changes
const ThemeContext = createContext<Theme>("light"); // rarely changes
```

2. Optimizing before you measure (premature optimization)

`React.memo`, `useMemo`, and `useCallback` are **not free**. They cost memory and add code you have to read. Sprinkling them everywhere "just in case" usually makes things slower *and* harder to understand.

- ❌ Wrap every component in `React.memo` and every value in `useMemo` before there's any real slowness.

- ☒ Write the simple version first. If something feels slow, **measure** with React DevTools Profiler, then optimize the specific slow part.

```
// ❌ pointless - adding two numbers is already instant
const total = useMemo(() => a + b, [a, b]);

// ✅ just do it
const total = a + b;
```

Rule of thumb: `useMemo` / `useCallback` earn their keep for **expensive** calculations, or to keep a prop **stable** for a memoized child (see #3). Not for cheap math.

3. `React.memo` that never actually helps

`React.memo` skips a re-render **only if the props are the same** as last time. React compares props by reference (`===`). If you create a **new** object, array, or function on every render, memo sees "different props every time" and never skips.

- ❌ Pass an inline object/array/arrow function straight into a memoized child.
- ✅ Wrap objects/arrays in `useMemo` and functions in `useCallback` so the reference stays stable.

```
const Row = React.memo(function Row(props: { style: React.CSSProperties }) {
  return <div style={props.style}>employee row</div>;
});

// ❌ new {} object every render → memo never skips
<Row style={{ color: "red" }} />;

// ✅ stable reference → memo can skip when nothing changed
const rowStyle = useMemo<React.CSSProperties>(() => ({ color: "red" }), []);
<Row style={rowStyle} />;
```

Same idea for callbacks: `onClick={() => save()}` is a brand-new function each render — wrap it in `useCallback`.

4. Wrong or missing dependency arrays

The array `[ ... ]` in `useMemo` / `useCallback` (and `useEffect` from Day 5) tells React: "recompute only when one of these changes."

- ❌ Leave a value out of the array → you keep an **old (stale)** value. It silently uses last render's data.

- ❌ Put a **new** object/array in the deps every render → it recomputes every time, so the memo does nothing.
- ✅ List **every** value from inside the callback that comes from props/state. Let the ESLint `react-hooks/exhaustive-deps` rule check you.

```
// ❌ uses `filter` but doesn't list it → stale results when filter changes
const visible = useMemo(() => employees.filter(matches(filter)), [employees]);

// ✅ list everything the calculation reads
const visible = useMemo(() => employees.filter(matches(filter)), [employees, filter]);
```

5. Mutating state inside a reducer

A reducer must be **pure**: given the same input, return a **brand-new** state object. Never change the old state in place. React decides whether to re-render by checking if the reference changed — mutate in place and you return the *same* reference, so the UI may not update.

- ❌ `state.count++` or `state.list.push(x)` then `return state`.
- ✅ Return a new object/array with the spread (`...`) operator.

```
// ❌ mutates the old state and returns the SAME reference
case "increment":
  state.count += action.by; // don't do this
  return state;

// ✅ return a fresh object
case "increment":
  return { ...state, count: state.count + action.by };
```

C#/.NET link: treat state like an **immutable record** — you produce a new one with `with { ... }`, you don't edit the old instance.

6. Expecting error boundaries to catch everything

An error boundary (a class component with `getDerivedStateFromError` + `componentDidCatch`) only catches errors thrown **during rendering** (and in lifecycle methods) of the components **below** it. It does **not** catch:

- errors inside **event handlers** (`onClick`, `onSubmit`, ...)
- errors in **async** code (`setTimeout`, promises, `await`, data fetches)

- errors thrown in the boundary itself
- ❌ Assume a boundary will show a nice fallback when your Save button's click handler throws.
- ✅ Use plain `try/catch` for event handlers and async code, and set an error into state yourself.

```
function SaveButton() {
  const [error, setError] = useState<string | null>(null);

  const handleClick = async () => {
    try {
      await saveEmployee();          // assume this is defined elsewhere
    } catch {
      setError("Could not save. Please try again.");
      // an error boundary would NOT catch this throw - it's async/event code
    }
  };

  return (
    <
      <button onClick={handleClick}>Save</button>
      {error && <p role="alert">{error}</p>}
    </>
  );
}
```

The `react-error-boundary` library gives you a ready-made boundary plus a `useErrorBoundary` helper — nice, but it follows the same rule: render errors only.

7. Forgetting `<Suspense>` around a `React.lazy` component

While a lazy component's code chunk is downloading, it literally **throws** (a special promise) so React knows to show a fallback. If there's no `<Suspense>` above it, that throw has nowhere to land and your app crashes.

- ❌ Use `React.lazy( ... )` with no `<Suspense>` wrapper.
- ✅ Wrap it (once, somewhere above it) in `<Suspense fallback={ ... }>`.

```
const EmployeeReport = React.lazy(() => import("./EmployeeReport"));

// ❌ nothing to show while it loads → it throws
// <EmployeeReport />

// ✅ Suspense provides the "loading..." fallback
<Suspense fallback={<p>Loading report...</p>}>
  <EmployeeReport />
</Suspense>;
```

8. Using `useContext` without a matching Provider

`useContext` walks **up** the tree looking for the nearest `Provider`. If there isn't one, you silently get the **default value** you passed to `createContext` — which is often wrong and hides the bug.

- ❌ Read a Context somewhere that isn't wrapped in its `Provider`, and quietly get a fake default.
- ❌ Give the Context a "realistic-looking" default just to stop TypeScript complaining — it hides the missing Provider.
- ✅ Default to `null` and use a small custom hook that **throws a clear error** when the Provider is missing.

```
const UserContext = createContext<User | null>(null);

// ✅ fails loudly and early instead of returning a fake user
function useUser(): User {
  const user = useContext(UserContext);
  if (user === null) {
    throw new Error("useUser must be used inside <UserProvider>");
  }
  return user;
}
```

9. Thinking `useMemo` / `useCallback` "cache forever"

They only remember the value **until a dependency changes**. Change a dep and it recomputes. Also — React is allowed to **throw the cache away** to free memory. So they are a **performance hint**, not a guarantee, and definitely not permanent storage.

- ❌ Rely on `useMemo` to run something "only once ever" or to store real app data.
- ✅ Treat it as: "skip this recompute *when the inputs are the same*." For true one-time setup, use `useEffect` (Day 5) or lazy `useState` init.

```
// runs again every time `query` OR `employees` changes – not "once forever"
const results = useMemo(() => search(employees, query), [employees, query]);
```

10. A reducer with an untyped (or loosely-typed) Action

If your action is just `{ type: string; payload?: any }`, TypeScript can't help you. A typo like `"inremnt"` compiles fine, and `payload` could be anything. Use a **discriminated union** (Day 2!) — one shape per action `type`. Then TS auto-completes the types and narrows `action` inside each `case`.

- ❌ `type Action = { type: string; payload?: any }` — typos and wrong payloads slip through.
- ✅ One object shape per action, joined by `|`. In `switch (action.type)`, TS knows exactly which fields exist.

```
type CounterState = { count: number };

// ✅ discriminated union: each action has its own typed payload
type CounterAction =
  | { type: "increment"; by: number }
  | { type: "reset" };

function reducer(state: CounterState, action: CounterAction): CounterState {
  switch (action.type) {
    case "increment":
      return { ...state, count: state.count + action.by }; // `by` is known to be a number
    case "reset":
      return { count: 0 };
  }
}
```

C#/.NET link: this is like a `switch` over an **enum**-driven set of command records — the compiler forces every command to carry exactly the right data.

One-line summary: lift state before you reach for Context, split contexts by change-rate, measure before you memoize, keep memo props stable, keep reducers pure and typed, and remember error boundaries + Suspense only handle **render-time** things — everything else needs `try/catch`.

4 Concept Notes — Recall & Interview (copy by pen)

Quick, point-to-point notes to recall Day 8 (Advanced React) concepts and answer classic interview questions — meant to be copied by pen, not read as paragraphs.

Sharing State Without Prop Drilling

Prop Drilling

Definition:

- Passing a prop down through many components that don't use it, just to reach a deep child.

✗ Disadvantages / watch-outs:

- Middle components get cluttered with props they don't care about.
- One rename means editing every component in the chain.

💡 Interview tip:

- Classic: "How do you avoid prop drilling?" → Context API (or a state library like Redux/Zustand).

📄 Example:

- `<App user={u}>` → `<Layout user={u}>` → `<Nav user={u}>` — same prop hand-carried down 3 levels.

Context API

Definition:

- A built-in way to share a value with any component in a subtree, without passing props by hand.

✅ Advantages:

- No prop drilling — deep children read the value directly.
- Great for "global-ish" data: current user, theme, language.
- C#/.NET: Context ~ dependency injection / a shared service you register once and read anywhere.

✗ Disadvantages / watch-outs:

- Every consumer re-renders when the context value changes.
- Not a full state manager — no built-in dev tools, middleware, or time-travel.

💡 Interview tip:

- Classic: "Context vs Redux vs props?" → props for local, Context for a few shared values, Redux for large/complex app state with tooling.

📄 Example:

- `const UserContext = createContext<User | null>(null)` — a typed shared box.

Provider

Definition:

- The component that wraps a subtree and supplies the current context value to everything inside it.

✓ Advantages:

- One place to hold and update the shared value.

✗ Disadvantages / watch-outs:

- Wrap only what needs it — a value change re-renders all consumers below.
- Pass a stable/memoized value object so you don't trigger re-renders for no reason.

📄 Example:

- `<UserContext.Provider value={user}>...</UserContext.Provider>` — makes `user` available below.

useContext

Definition:

- The hook that reads the nearest Provider's value inside a component.

✓ Advantages:

- One clean, typed line to grab shared data.

✗ Disadvantages / watch-outs:

- Returns the default value if there is no Provider above — wrap it in a custom hook that throws, so you catch the mistake early.

📄 Example:

- `const user = useContext(UserContext)` — reads the shared user.

Managing Complex State

useReducer

Definition:

- A hook for state where the next value depends on the current one and updates arrive as clearly "typed" actions.
- The 3 parts: **reducer** = pure function `(state, action) ⇒ newState`; **action** = an object like `{ type: 'HIRE' }`; **dispatch** = the function you call to send an action.

✓ Advantages:

- Keeps update logic in one place (the reducer), not scattered across handlers.
- Easy to test — the reducer is a pure function.
- Pairs perfectly with TypeScript discriminated unions for the action type (Day 2 pays off).
- C#/.NET: useReducer ~ a Redux-style store / a small state machine.

❌ Disadvantages / watch-outs:

- Overkill for one or two simple, independent values — `useState` is simpler there.

💡 Interview tip:

- Classic: "useReducer vs useState?" → `useState` for simple independent values; `useReducer` for complex/related state or many action types.

📄 Example:

- `const [state, dispatch] = useReducer(reducer, initial)` — state plus a dispatcher.

State machine idea

Definition:

- Thinking of state as a set of allowed "states" that you only move between via named actions.

✅ Advantages:

- Makes impossible states impossible (e.g., can't be "loading" and "error" at once).
- Reducer + action types map naturally to this idea.
- C#/.NET: like an enum-driven workflow / state machine you might model in C#.

💡 Interview tip:

- Good line: "Reducers encourage explicit transitions, which prevents invalid UI states."

📄 Example:

- `idle → loading → success | error` — named transitions, not random boolean flags.

Performance & Memoization

Referential equality

Definition:

- React compares objects, arrays, and functions by their reference (identity), not by their contents.

✅ Advantages:

- Cheap and fast — just checks "same object in memory?".
- C#/.NET: like reference equality vs value equality (`ReferenceEquals` vs `.Equals`).

❌ Disadvantages / watch-outs:

- A new object/array/function literal each render is a brand-new reference, even if it looks identical.
- This is exactly why children re-render and why `useMemo`/`useCallback` exist.

💡 Interview tip:

- Classic: "Why does my memoized child still re-render?" → you passed a fresh object/function reference every render.

Example:

- `a === b` where `a = {}` and `b = {}` is `false` — same shape, different reference.

Memoization

Definition:

- Caching the result of a computation so you can reuse it instead of redoing the work.

✓ Advantages:

- Skips expensive recalculation when the inputs haven't changed.
- C#/.NET: memoization ~ caching a computed value (like a cached property or `Lazy<T>`).

✗ Disadvantages / watch-outs:

- Has its own cost (memory + comparison) — it is not free, so don't sprinkle it everywhere.

Example:

- `useMemo(() => expensiveCalc(x), [x])` — recompute only when `x` changes.

React.memo

Definition:

- A wrapper that skips re-rendering a component if its props are the same (by reference) as last time.

✓ Advantages:

- Stops a child re-rendering when the parent renders but the child's props didn't change.

✗ Disadvantages / watch-outs:

- Only helps if props are referentially stable — often needs `useMemo/useCallback` on those props too.
- Wasteful on cheap components or ones that always get fresh props.

💡 Interview tip:

- Classic: "What does `React.memo` do?" → memoizes a component's output using a shallow compare of its props.

Example:

- `const Row = React.memo(EmployeeRow)` — Row skips render when its props are unchanged.

useMemo

Definition:

- A hook that caches a computed value and only recomputes it when its dependencies change.

✓ Advantages:

- Avoids re-running expensive calculations on every render.
- Keeps an object/array reference stable so memoized children don't re-render.

❌ Disadvantages / watch-outs:

- Don't use it for cheap values — the caching costs more than the calc.

Example:

- `const sorted = useMemo(() => [...list].sort(cmp), [list])` — re-sorts only when `list` changes.

useCallback

Definition:

- A hook that caches a function so it keeps the same reference between renders (until its deps change).

✅ Advantages:

- Keeps callback props stable, so React.memo children don't re-render.

❌ Disadvantages / watch-outs:

- Pointless unless the function is passed to a memoized child or used in a dependency array.

Example:

- `const onHire = useCallback(() => hire(id), [id])` — same function reference until `id` changes.

useCallback vs useMemo

Definition:

- `useCallback` caches a **function**; `useMemo` caches a **value**. In fact `useCallback(fn, d)` equals `useMemo(() => fn, d)`.

💡 Interview tip:

- Classic: "useMemo vs useCallback?" → `useMemo` returns the *result* of calling the function; `useCallback` returns the *function itself*.

Example:

- `useMemo(() => x*2, [x])` returns a number; `useCallback(() => x*2, [x])` returns a function.

When to optimize / premature optimization

Definition:

- Add memoization only after you measure a real performance problem — not by default.

✅ Advantages:

- Keeps code simple and readable; avoids fake "optimizations" that don't actually help.

❌ Disadvantages / watch-outs:

- Wrapping everything in memo/useMemo/useCallback adds noise and can even be slower.
- Measure first (React DevTools Profiler), then optimize the real hotspot.

💡 Interview tip:

- Good line: "Premature optimization adds complexity for gains you haven't proven you need."

Loading Code Lazily (Code Splitting)

React.lazy

Definition:

- A function that loads a component's code only when it is first rendered, instead of upfront.

✅ Advantages:

- Smaller initial bundle — faster first load.

❌ Disadvantages / watch-outs:

- Must be rendered inside a `<Suspense>` boundary, or it throws.
- Works with default exports (or re-export as default).

📄 Example:

- `const Reports = React.lazy(() => import('./Reports'))` — load Reports on demand.

Suspense

Definition:

- A component that shows a fallback (like a spinner) while lazy content is still loading.

✅ Advantages:

- Clean, declarative loading UI — no manual "isLoading" flag just to load code.

📄 Example:

- `<Suspense fallback={<Spinner/>}><Reports/></Suspense>` — spinner until Reports arrives.

Code splitting

Definition:

- Breaking your app's JavaScript into smaller chunks that load only when they're needed.

✅ Advantages:

- Users download less on the first visit; rarely used pages load later.

💡 Interview tip:

- Classic: "How do lazy/Suspense enable code splitting?" → `import()` creates a separate bundle chunk; React.lazy loads it on demand and Suspense shows a fallback meanwhile.

📄 Example:

- Route-level: lazy-load each page so the dashboard doesn't ship the whole app at once.

Resilience: Error Boundaries

Error boundary

Definition:

- A class component that catches render-time errors in its child tree and shows a fallback UI instead of a blank/crashed screen.

✅ Advantages:

- Stops one broken component from crashing the whole app.
- A central place to log errors and show a friendly "something went wrong".
- C#/.NET: like a try/catch around the UI tree — but only for rendering, not for every operation.

❌ Disadvantages / watch-outs (what it does NOT catch):

- Event handler errors (use try/catch there).
- Async code (setTimeout, promises, fetch callbacks).
- Server-side rendering errors.
- Errors thrown inside the boundary itself.

💡 Interview tip:

- Classic: "What do error boundaries catch?" → errors during render, in lifecycle methods, and in constructors of the tree below — NOT events or async.
- `getDerivedStateFromError` → set state to show the fallback (render phase); `componentDidCatch` → log the error/details (side effects).

📄 Example:

- `static getDerivedStateFromError() { return { hasError: true } }` — flip to fallback UI.
- Note: this is the one legit modern use of a class component (or use the `react-error-boundary` library).